缓存击穿穿透雪崩

缓存击穿

缓存击穿就是说, 一个被高并发访问的key突然失效了，大量请求访问数据库会带来非常大的压力.

- 缓存击穿有很多种方案, 甚至有公司直接搞了一个中间件去解决这个问题

互斥锁方案

互斥锁方案就是加锁, 多个请求同时访问, 拿到锁的请求才能去访问数据库去更新缓存, 拿不到锁的请求就阻塞等待.

然后做一个双重检测锁的机制, 等锁释放, 其他请求拿到锁后先检查缓存中有没有数据, 有数据直接返回就行了.

总之就是保证只能有一个请求访问数据库去更新缓存. 这种互斥锁的方案, 数据一致性强, 但是要阻塞等待, 性能没那么好.

逻辑过期方案

逻辑过期就是缓存设置永不过期, 然后数据内部维护一个过期时间

请求来了, 拿到数据, 判断数据内部的过期时间是否过期了, 过期了再抢锁.

抢到锁的线程去开启一个新线程去异步更新缓存. 其他请求直接返回旧数据.

这种方案, 数据一致性差, 但是不用阻塞等待性能更好.

CacheSetter

某手, 有个rpc服务叫cache setter

通过一致性hash, 让key相同的请求都路由到同一个cache setter.

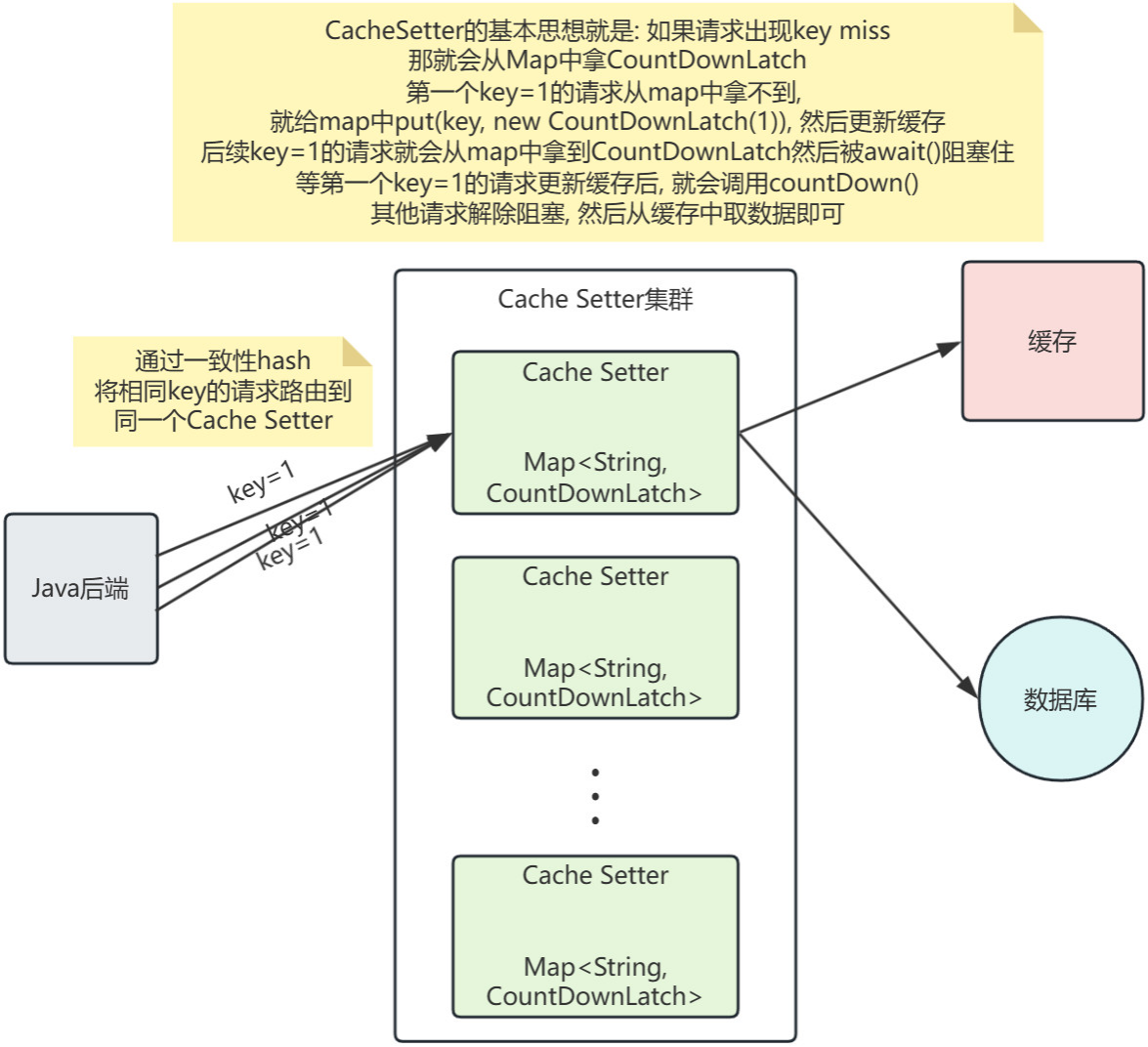
cache setter内部维护了一个map, map的value是CountDownLatch.

出现缓存查不到的情况, 就从map中尝试拿CountDownLatch, 拿不到就说明这是第一个key miss的请求, 那么就会给map中put一个CountdownLatch, 然后去更新缓存.

后续请求过来, 就会从map中拿到一个CountdownLatch然后被await()方法阻塞住.

当第一个请求更新完缓存后, 就调用countdown()方法, 然后其他线程就解除阻塞了, 就直接从缓存中拿数据即可.

- 这种方案和互斥锁的方案很像, 只是它不是通过分布式锁阻塞请求的, 而是通过一致性hash和map中的countDownLatch来阻塞相同key的请求, 保证只有一个请求访问数据库.



jd hotkey

jd hotkey, 通过检测出hotkey然后直接将hot key放入本地缓存去解决问题.

缓存穿透

缓存穿透就是说请求的数据在数据库和缓存中都不存在, 那么请求就会直接访问数据库, 数据库造成很大的压力.

- 首先, 对请求参数做好校验, 直接拦截一些非法请求. 对于异常用户, 或者异常ip, 直接做限流, 或者设置黑名单. 尽量在网关或者在请求检查这一步就把非法请求拦住.

- 然后就是缓存空值, 对于数据库中查不到的key, 缓存一个空数据, 设置一个比较短的过期时间, 避免后面增加了这个数据反而查不到.
- 但是如果每个请求查的key都不一样, 那缓存空值就失效了. 这时候可以上布隆过滤器. 写数据库的时候去同时写布隆过滤器, 然后请求来的时候通过布隆过滤器快速判断数据是否存在.
    - 简单说一下布隆过滤器的原理, 就是把数据经过多个hash函数进行计算, 计算出多个hash值, 然后把hash值映射到一个bitmap中, 把对应位置置1. 请求来了, 就对这些数据用hash函数做计算, 然后看看这几个hash值对应的bitmap的那个位置上是不是1, 是1说明有, 是0说明没有.
    - 但是布隆过滤器缺点很大, 一个就是有有误判的可能, 另一个就是布隆过滤器不支持删除, 如果数据库对数据做了删除操作, 布隆过滤器是没办法同步删除数据的.

缓存雪崩

缓存雪崩是指在同一时段大量的缓存key同时失效或者Redis宕机了，导致大量请求到达数据库，导致数据库宕机

- 对于大量key同时过期
    - 第一种方案就是在固定过期时间的基础上去加上一个随机数, 让这些数据不会在同一时间过期, 那就能一定程度上避免同一时间大量key过期
    - 那第二种方案针对于那些几乎不变化的数据不设置过期时间, 只要我不设置过期时间那就不会同时过期. 然后修改这些数据的异步地去删除/更新缓存就可以了.
    - 第三种方案是针对那些能够预知到热点数据, 做缓存预热, 保证在业务高峰时间不会过期.

- 那么针对redis宕机了, 或者内存不够了导致数据一直被淘汰, 或者缓存中有大量冷数据进来, 热点数据被逐出的, 或者就是单纯不可抗力, 比如机房被洪水淹了. 这种不可控的意外情况也会导致缓存雪崩问题.
    - 针对redis宕机, 就是做好redis高可用, 该做主从做主从, 该上集群上集群
    - 然后就是要针对一些不可控的意外做好兜底措施, 万一真的发生缓存雪崩, 做好服务降级限流熔断这种服务保护措施, 避免服务被打垮.